Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Latency, Throughput, Bandwidth and Jitter

Setting: Data travels from a sender to a receiver: packets over a network, requests to an API, tokens out of an LLM. Four metrics describe how well that happens, and they are often confused:

- **Latency:** *how long* one unit of data takes to arrive. Unit: time (ms).
- **Bandwidth:** the *maximum* amount of data the channel can carry per unit of time. Unit: bits/s (Mbps, Gbps). It is a capacity.
- **Throughput:** the amount of data (or requests) *actually* delivered per unit of time. Unit: bits/s, requests/s, tokens/s. It is a measurement.
- **Jitter:** how much the latency *varies* from one unit to the next. Unit: time (ms).

The pipe analogy: think of a water pipe. **Bandwidth** is the pipe's *width*; **latency** is the time a drop takes to go from one end to the other (the pipe's *length*); **throughput** is how much water really comes out per second (it can be less than the width allows, if there are leaks or the tap is half open); **jitter** is how irregular the arrival times are.

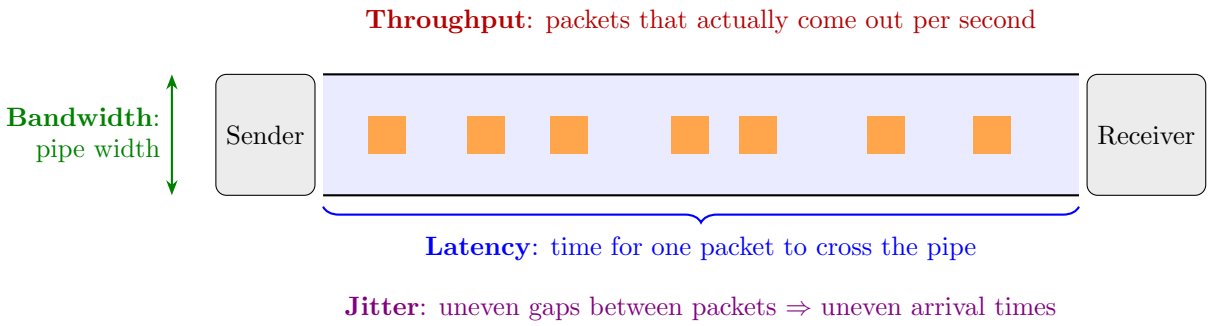


Figure 1: Bandwidth is a property of the channel; latency, throughput and jitter are observed while data flows.

Key relations:

$$\text{Throughput} \leq \text{Bandwidth}, \qquad \text{Latency and bandwidth are independent}$$

A satellite link can have huge bandwidth and terrible latency (the signal travels $\sim 36{,}000$ km up and back); a short cable between two servers can have tiny latency and modest bandwidth. Adding bandwidth does **not** reduce the latency of a single small request.

1. Latency

The total delay for one packet is the sum of four components:

$$\text{Latency} = \underbrace{\frac{d}{v}}_{\text{propagation}} + \underbrace{\frac{L}{B}}_{\text{transmission}} + \underbrace{t_{queue}}_{\text{queuing}} + \underbrace{t_{proc}}_{\text{processing}}$$

- **Propagation** $\frac{d}{v}$: distance d divided by signal speed v (in fiber $v \approx 2 \times 10^5$ km/s, about $\frac{2}{3}$ of the speed of light). It cannot be improved by buying hardware, only by getting closer (CDNs, edge regions).
- **Transmission** $\frac{L}{B}$: time to push a packet of L bits onto a link of bandwidth B. This is the only term where bandwidth helps.
- **Queuing:** waiting in router or server buffers when traffic exceeds capacity. It is the most variable term, and the main source of jitter.
- **Processing:** time spent by routers, the OS, the application, or the model inference.

Example: Santiago → Virginia (US East), $d \approx 8{,}000$ km:

$$t_{prop} = \frac{8{,}000 \text{ km}}{2 \times 10^5 \text{ km/s}} = 40 \text{ ms (one way)} \;\Rightarrow\; RTT \geq 80 \text{ ms}$$

Real pings are around 120–150 ms because cables are not straight and add queuing and processing.

RTT (Round-Trip Time) is the latency of going and coming back; it is what `ping` measures. A request that needs several round trips (TCP handshake, TLS handshake, then the HTTP request) pays the RTT several times.

Measure percentiles, not the mean. Latency distributions have long tails. Report **p50** (median, typical user), **p95** and **p99** (the slowest 5% and 1%). If a page makes n independent calls, the probability that at least one hits the p99 tail is:

$$P(\text{at least one slow call}) = 1 - 0.99^n, \qquad n = 100 \;\Rightarrow\; 1 - 0.99^{100} \approx 63\%$$

so tail latency, not average latency, decides user experience at scale.

2. Bandwidth

The maximum rate of a link, fixed by the physical medium and the contract: e.g. a 1 Gbps Ethernet port, a 500 Mbps home fiber plan, a 25 Gbps network on a cloud instance. Note the units: network speeds use **bits** (b), file sizes use **bytes** (B), and 1 B = 8 b:

$$500 \text{ Mbps} = \frac{500}{8} = 62.5 \text{ MB/s} \;\Rightarrow\; \text{a 10 GB file needs at least } \frac{10{,}000}{62.5} = 160 \text{ s}$$

Bandwidth-Delay Product (BDP): how much data is "in flight" in the pipe at any moment:

$$BDP = B \cdot RTT$$

For $B = 1$ Gbps and $RTT = 100$ ms: $BDP = 10^9 \cdot 0.1 = 10^8$ bits = 12.5 MB. To fill that pipe, the sender must keep 12.5 MB unacknowledged in flight.

3. Throughput

What you actually get. It is lower than the bandwidth because of protocol overhead (headers), packet loss and retransmissions, congestion, slow endpoints, and the latency itself. For TCP, the sender can only have one window W of unacknowledged data per round trip:

$$\text{Throughput}_{TCP} \;\le\; \min\left(B,\; \frac{W}{RTT}\right)$$

Example: with a 64 KB window and $RTT = 100$ ms: $\frac{64 \text{ KB}}{0.1 \text{ s}} = 640$ KB/s ≈ 5 Mbps, even on a 1 Gbps link. This is why **latency limits throughput** on long-distance transfers, and why modern TCP scales the window up to the BDP.

Throughput in systems (not only networks): for a server, throughput is requests/s; for an LLM, tokens/s. **Little's Law** links it with latency and concurrency:

$$L = \lambda \cdot W \qquad \Longleftrightarrow \qquad \underbrace{\lambda}_{\text{throughput}} = \frac{\overbrace{L}^{\text{requests in progress}}}{\underbrace{W}_{\text{latency}}}$$

A service handling $L = 50$ concurrent requests with $W = 200$ ms latency serves $\lambda = \frac{50}{0.2} = 250$ requests/s. To increase throughput you either reduce latency or increase concurrency.

4. Jitter

Jitter is the variability of latency between consecutive packets. If packet i is sent at S_i and received at R_i, its transit time is $D_i = R_i - S_i$. A simple definition is the mean absolute difference between consecutive delays:

$$J = \frac{1}{n-1} \sum_{i=2}^{n} |D_i - D_{i-1}|$$

Real-time protocols (RTP, RFC 3550) use a smoothed running estimate, updated with every packet:

$$J_i = J_{i-1} + \frac{|D_i - D_{i-1}| - J_{i-1}}{16}$$

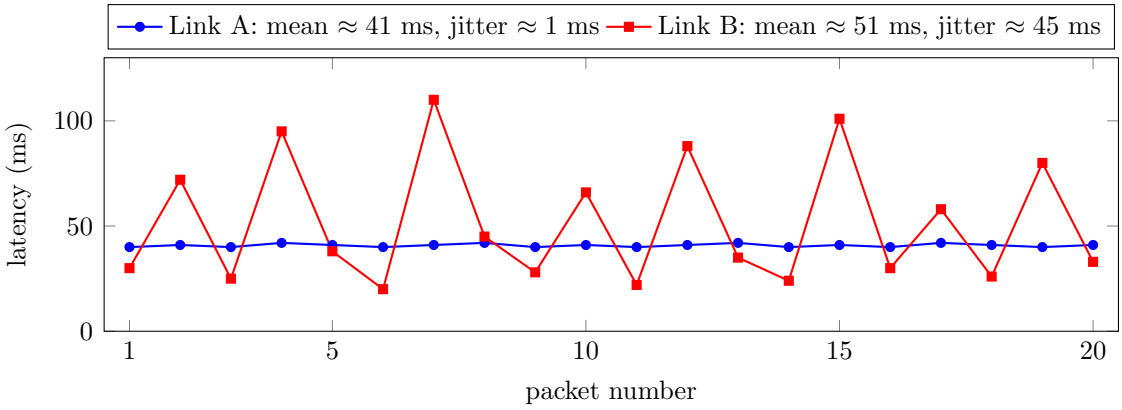


Figure 2: Similar average latency, very different jitter. Link A is far better for a video call.

Why it matters: file downloads barely care about jitter, but real-time traffic does. In a video call, audio packets must be played every 20 ms; if some arrive late, the receiver hears gaps. The fix is a **jitter buffer**: hold packets a few ms before playing them, smoothing arrivals at the cost of *adding* latency. Typical targets for voice: latency < 150 ms one way, jitter < 30 ms, loss $< 1\%$.

Causes: variable queuing in congested routers, Wi-Fi interference and retransmissions, packets taking different routes, CPU contention on the endpoints.

How to measure all four

Tool	Measures
`ping`	RTT (latency); min / avg / max / mdev of many pings gives a quick jitter estimate
`traceroute`, `mtr`	Latency hop by hop: where the delay or loss appears
`iperf3`	Throughput between two hosts (TCP); in UDP mode also jitter and packet loss
Speed tests	Download / upload throughput and latency to a nearby server
Load testing (k6, Locust, wrk)	Application latency percentiles (p50, p95, p99) and requests/s
Prometheus + Grafana	Latency histograms and throughput over time in production

In ML systems: serving a model or an LLM

The same four concepts appear when serving models:

- **Latency:** time per request. For LLMs it splits into **TTFT** (time to first token, dominated by processing the prompt) and **TPOT / ITL** (time per output token, the generation speed seen by the user):
$$\text{Total latency} = TTFT + (n_{tokens} - 1) \cdot TPOT$$
- **Throughput:** requests/s or total tokens/s across all users.
- **Bandwidth:** GPU memory bandwidth (GB/s) is the real limit for LLM decoding: each generated token requires reading all model weights from memory. Roughly, for a model of size M bytes on a GPU with memory bandwidth B_{mem}:
$$TPOT_{min} \approx \frac{M}{B_{mem}}, \qquad \text{e.g. } \frac{\text{16 GB (8B params, fp16)}}{\text{2,000 GB/s}} \approx 8 \text{ ms per token}$$
- **Jitter:** variation in response time between requests, caused by different prompt lengths, batching, cold starts and noisy neighbors; it makes streaming feel choppy.

The batching trade-off: grouping requests in a batch increases throughput (the weights are read once for many requests) but each request waits longer, so latency increases.

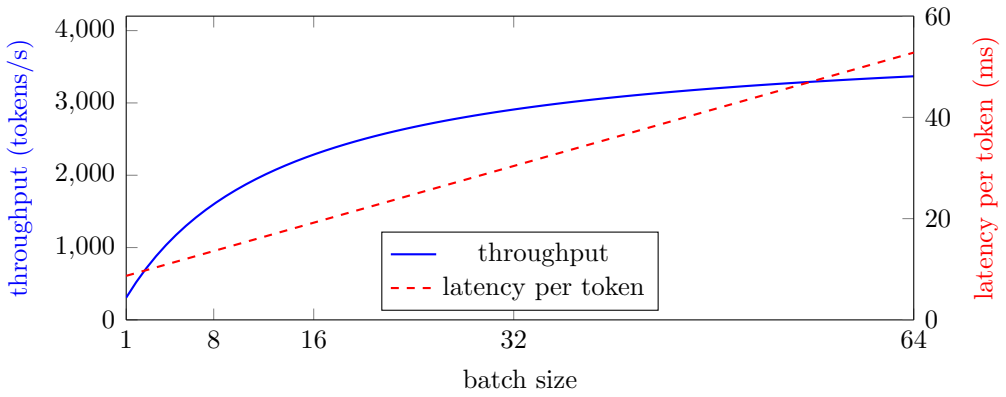


Figure 3: Illustrative shape: throughput saturates as batch size grows, while latency keeps rising. The operating point is chosen by the latency SLO.

Comparison:

	Latency	**Bandwidth**	**Throughput**	**Jitter**
Question	How long does one unit take?	How much can the channel carry at most?	How much is actually delivered?	How stable is the latency?
Unit	ms	bits/s	bits/s, req/s, tokens/s	ms
Nature	Measured delay	Capacity (theoretical)	Measured rate	Variability of latency
Better when	Lower	Higher	Higher	Lower
Matters most for	Interactive apps, APIs, gaming, trading	Large transfers, streaming video	Batch jobs, data pipelines, serving capacity	Voice, video calls, live streaming
Main fix	Get closer (CDN, edge), fewer round trips	Upgrade link / instance	Remove bottlenecks, parallelize, batch, tune windows	Reduce congestion, QoS, jitter buffers

Bandwidth is how wide the pipe is; **throughput** is how much really flows through it ($\leq$ bandwidth); **latency** is how long one drop takes to cross it; **jitter** is how irregular those crossing times are. More bandwidth does not fix latency, high latency can cap throughput ($\frac{W}{RTT}$), and a link with good average latency can still be unusable for real-time traffic if its jitter is high. Always measure latency by percentiles (p50, p99), not by the mean.